شرح تفصيلي لأدوار الفريق في مشروع Academic Intelligence System

مقدمة

الملف ده معمول بحيث يشرح بشكل واضح وعميق دور كل عضو في الفريق داخل مشروع `Academic Intelligence System` ، وخصوصًا من زاوية:

- كل عضو بنى إيه فعلًا
- بنى الجزء ده إزاي من الناحية التقنية
- ليه التصميم ده اتختار بالشكل ده
- الجزء ده بيتكامل إزاي مع باقي أعضاء الفريق
- إيه الـ execution flow الحقيقي وقت تشغيل النظام

الشرح هنا مناسب جدًا للاستخدام في:

- مناقشة المشروع
- الـ viva
- التقرير النهائي
- شرح الـ system design أمام اللجنة

Member 1: Frontend Lead / Dashboard & User Experience

1. What He Built

عضو رقم 1 كان مسؤول بشكل أساسي عن الواجهة الرئيسية التي يتعامل معها الطالب داخل النظام، وخصوصًا الجزء الخاص بتجربة الاستخدام العامة `User Experience` .

هو بنى ونسّق الأجزاء التالية:

- الصفحة الرئيسية للطالب داخل الداشبورد
- الـ layout العام للنظام
- الـ sidebar الخاصة بالتنقل بين الصفحات
- الـ header العلوي

- عرض التنبيهات والإشعارات
- ربط الـ visual components ببيانات APIs الجاهزة
- توحيد شكل وتصميم النظام في كل الصفحات

بمعنى أوضح: هذا العضو لم يكن مسؤولًا عن منطق حساب البيانات نفسها، لكنه كان مسؤولًا عن تحويل البيانات الجاهزة إلى تجربة استخدام واضحة، سهلة، ومفهومة للطالب.

2. How He Built It (Technical Deep Dive)

من الناحية التقنية، هذا العضو اعتمد على:

- `Next.js` كإطار عمل للواجهة
- `React` لبناء الـ components
- `Tailwind CSS` أو أسلوب styling مشابه لبناء واجهة منظمة وسريعة
- مكونات مشتركة مثل:
  - `Header.tsx`
  - `Sidebar.tsx`
  - `Loading.tsx`
  - `animations.tsx`

القرار المعماري المهم هنا كان فصل:

- `presentation layer`
- عن
- `business logic layer`

نفسها، لكن components داخل الـ recommendation logic أو freshness أو منطق CGPA لم يضع حسابات Member 1 يعني وجهّزها بصريًا APIs استقبل البيانات من.

الخطوات التقنية التي اتبعها كانت غالبًا كالتالي:

1. تجهيز الـ dashboard shell بحيث يكون ثابت في كل الصفحات.
2. بناء navigation system واضح للطالب.
3. ربط صفحة الداشبورد مع API مثل `/api/dashboard`.
4. استقبال response يحتوي على:
   - بيانات المستخدم
   - بيانات أكاديمية
   - progress
   - عدد المواد الحالية
   - تنبيهات أو نشاط حديث
5. تحويل هذه البيانات إلى cards وsections واضحة.
6. إضافة حالات `loading`، `empty`، و `error` حتى لا تبدو الصفحة مكسورة لو البيانات لم تصل.

فقط، بل كمنصة أكاديمية متكاملة. وده اتعمل عن طريق AI وهو أن النظام لا يظهر كأداة UX، هو أيضًا مسؤول عن قرار مهم جدًا في الـ:

- وضع overview cards
- ربط الإشعارات والتنبيهات داخل الداشبورد
- جعل الوصول لـ Profile وStudy Chat وPlanner سهل ومباشر

3. Real Flow Example

مثال عملي حقيقي:

عندما يفتح الطالب صفحة الداشبورد:

1. المتصفح يدخل على صفحة `/dashboard`.
2. Next.js والـ layout العام يقوم بتحميل صفحة الداشبورد.
3. الواجهة ترسل طلب إلى `/api/dashboard`.
4. الـ API يعيد بيانات الطالب:
    - الاسم
    - CGPA
    - المواد الحالية
    - نسبة التقدم نحو التخرج
5. Member 1's UI layer تستقبل الـ response.
6. يتم توزيع البيانات على:
    - CGPA خاصة بالـ card
    - للمواد الحالية card
    - لنسبة التقدم card
    - للتنبيهات section
7. الطالب يرى لوحة معلومات جاهزة بدون الحاجة لفهم تفاصيل معقدة في الخلفية.

4. Integration With Other Members

هذا العضو يعتمد بشكل كبير على الأعضاء الآخرين، لكنه لا يكرر شغلهم.

البيانات التي يستقبلها:

- من Member 5:
    - dashboard APIs
    - profile freshness alerts
    - advisor insights
- من Member 2:
    - حالة البروفايل واكتمال البيانات
- من Member 3:
    - التنقل أو الربط مع Study Chat frontend

البيانات التي يرسلها أو يمررها:

- المناسبة APIs إلى user interactions بنفسه، لكن يمرر business decisions لا يرسل
- يوفّر نقاط دخول واضحة للصفحات الأخرى

بدل أن تبدو كموديولات منفصلة cohesive التكامل الحقيقي هنا أن هذا العضو جعل كل الأجزاء التي بناها باقي الفريق تظهر كمنتج واحد.

5. Why This Design

التصميم ده اتختار لأن:

- فصل الـ UI عن منطق الأعمال يجعل الصيانة أسهل
- أي تغيير في شكل الواجهة لا يفسد الـ APIs
- لا يحتاج إعادة بناء هيكل الواجهة بالكامل أي تعديل في الحسابات أو analytics

مميزات هذا الأسلوب:

- وضوح ownership
- تقليل التعارض بين أعضاء الفريق
- سهولة الاختبار
- سهولة تحسين الـ UX بدون لمس backend logic

Member 2: Academic Profile / Onboarding / Student Data Management

1. What He Built

عضو رقم 2 كان مسؤول عن الأساس الذي يعتمد عليه المشروع كله: بيانات الطالب الأكاديمية.

هذا العضو بنى:

- onboarding flow
- editable academic profile
- academic history management
- إدخال بيانات الطالب الأساسية
- إدخال السجل الأكاديمي semester by semester
- validation لمنع تكرار نفس المادة في نفس الترم
- حماية consistency بين البيانات المدخلة والـ database constraints

بمعنى عملي: هذا العضو هو الذي جعل النظام يعرف "الطالب مين" و"أخذ إيه" و"حاليًا مسجل إيه" و"ناوي ياخد إيه".

2. How He Built It (Technical Deep Dive)

تقنيًا اعتمد على:

- `Next.js` لصفحات الـ onboarding والـ profile
- component مركزي مثل `AcademicProfileEditor.tsx`
- API route مثل `/api/onboarding`
- `Supabase` للوصول إلى بيانات المستخدم وربطها بالجداول
- validation على مستوى الواجهة والـ API

القرار الأهم هنا كان أن `academic profile` لا يكون مجرد form بسيط، بل data model حقيقي مرتبط بباقي النظام.

الخطوات التقنية كانت تقريبًا:

1. تحميل البيانات الأساسية:
    - الجامعات
    - السمسترات
    - الكورسات
    - بيانات الطالب الحالية
2. عرض form يسمح بتحديث:
    - الاسم
    - القسم
    - الجامعة
    - total required hours
3. عرض جدول academic history بحيث كل row يحتوي على:
    - semester
    - course
    - status
    - grade
4. عند الحفظ:
    - يتم فحص الحقول الإلزامية
    - يتم منع duplicate rows
    - يتم إرسال payload واضح إلى `/api/onboarding`
5. في الـ API:
    - يتم التحقق من المستخدم الحالي
    - يتم تحديث profile
    - يتم إدخال أو تحديث `student_courses`
    - يتم التعامل مع unique constraints بشكل آمن

هذا العضو كان أيضًا مهم جدًا في تصحيح مشاكل مثل:

- duplicate key violations
- عدم الاتساق بين session data والمستخدم الحقيقي
- confusion بين catalog courses وstudent-linked courses

3. Real Flow Example

مثال واقعي:

لأول مرة onboarding عندما يفتح الطالب صفحة:

1. الصفحة ترسل `GET /api/onboarding` .
2. الـ API ترجع:
    - profile
    - universities
    - semesters
    - courses
    - studentCourses
3. الواجهة تعرض form جاهز.
4. الطالب يضيف مواده السابقة والحالية.
5. عند الضغط على Save:
    - يتم تكوين request body منسق
    - يرسل `POST /api/onboarding`
6. الـ backend يتحقق:
    - هل الطالب authenticated؟
    - هل هناك duplicate course in same semester؟
    - هل الحقول الأساسية مكتملة؟
7. يتم حفظ البيانات في الجداول المناسبة.
8. بعد ذلك تصبح هذه البيانات متاحة للداشبورد والـ planner والـ Study Chat.

4. Integration With Other Members

هذا العضو يرسل foundational data لكل النظام.

يعطي بيانات إلى:

- Member 5:
    - ليحسب dashboard analytics
    - ليحدد profile freshness
    - ليبني planner logic
- Member 3:
    - ليعرف المواد الحالية التي ستظهر في Study Chat
- Member 1:
    - ليعرض حالة الطالب في الواجهة

ويعتمد على:

- Supabase Auth لمعرفة user identity

- Member 5 لضمان API consistency

أي خطأ في هذا الجزء كان سيؤثر مباشرة على:

- dashboard
- study chat
- planner
- alerts

5. Why This Design

السبب في اختيار هذا التصميم هو أن كل features الذكية لاحقًا تحتاج قاعدة بيانات أكاديمية صحيحة.

لو profile وال academic history غير مضبوطين، سيفشل:

- التوصية
- التخطيط
- Study Chat course scoping
- progress calculation

مميزات التصميم:

- data integrity
- reusability across modules
- سهولة التحديث مع كل ترم جديد
- وضوح العلاقة بين الطالب ومواده وسجله الأكاديمي

Member 3: Study Chat / Course Learning Experience

1. What He Built

عضو رقم 3 بنى الجزء الذي يحول المشروع من academic dashboard إلى learning product حقيقي: `Study Chat`.

هذا العضو كان مسؤول عن:

- واجهة Study Chat
- اختيار المادة الحالية
- عرض مكتبة المصادر الخاصة بالمادة
- اختيار ملف أو عدة ملفات قبل السؤال
- إدارة جلسات الشات لكل مادة
- study modes مثل:
  - Chat

    - Summary
    - Quiz
    - Flashcards
    - Expected Questions
    - Study Plan
- عرض citations والمصادر تحت الردود

2. How He Built It (Technical Deep Dive)

اعتمد على:

- `Next.js`
- React state management داخل صفحة Study Chat
- endpoint مثل `/api/study/courses`
- backend chat endpoint الموثق كـ `/api/chat` والمنفذ فعليًا كـ `/api/ask`
- file-source selection logic في الواجهة

القرار المعماري المهم هنا هو أن Study Chat لا يكون general chat، بل `course-scoped chat`.

الخطوات التقنية:

1. تحميل المواد الحالية من `/api/study/courses`.
2. بمجرد اختيار المادة:
    - يتم تحميل study materials الخاصة بها
    - يتم عرض source library
3. الواجهة تتيح:
    - رفع ملفات
    - تحديد selected sources
    - اختيار mode
    - كتابة السؤال
4. عند الضغط على Send:
    - يتم إرسال payload يحتوي على:
        - `question`
        - `course_id`
        - `course_code`
        - `course_name`
        - `selected_document_ids`
        - `study_mode`
        - `conversation_id`
5. بعد وصول الرد:
    - يتم عرض answer

    - يتم عرض source cards
    - وروابط المصدر يتم عرض excerpts

هذا العضو اهتم أيضًا بالـ UX edge cases:

- ماذا يحدث لو لا توجد ملفات؟
- ماذا يحدث لو الـ token انتهى؟
- ماذا يحدث لو selected source واحد فقط؟
- ماذا يحدث لو الرد عاد بدون sources منظمة؟

يعني هو لم يبنِ مجرد chat box، بل study workflow كامل.

3. Real Flow Example

مثال حقيقي:

طالب فتح مادة `AI225` داخل Study Chat.

1. الواجهة تطلب المواد الحالية.
2. الطالب يختار AI225.
3. الصفحة تعرض:
    - الملفات المرفوعة للمادة
    - prompt suggestions
    - study modes
4. الطالب يحدد ملف `LEC1.pdf` .
5. يختار mode = `Summary` .
6. يكتب: "لخصلي أهم الأفكار في المحاضرات".
7. الواجهة ترسل request إلى backend.
8. بعد وصول الرد:
    - يتم عرض الملخص
    - تظهر citations تحت الإجابة
    - الطالب يقدر يفتح المصدر نفسه من الرابط

4. Integration With Other Members

Member 3 هو حلقة الوصل بين user learning experience وAI backend.

يعتمد على:

- Member 2:
    - لمعرفة المواد الحالية للطالب
- Member 4:
    - للحصول على grounded AI responses

    - citations
    - retrieval behavior
- Member 5:
    - للتأكد من API contracts والـ integration behavior

ويرسل إلى Member 4:

- course context
- selected documents
- study mode
- conversation context

ويستقبل منه:

- answer
- sources
- structured citations

5. Why This Design

هذا التصميم مهم لأنه يجعل التجربة:

- scoped
- explainable
- suitable for studying

لو كان Study Chat general-purpose فقط، كان سيعاني من:

- hallucination أعلى
- فقدان سياق المادة
- ضعف الثقة في الإجابات

مميزات هذا التصميم:

- كل مادة لها سياقها
- كل سؤال يمكن ربطه بمصدر
- النظام يصبح قريبًا من NotebookLM-style experience

Member 4: Backend AI / RAG / Document Processing

1. What He Built

يعتمد على ملفات حقيقية بدل أن يكون Study Chat نفسها، وخصوصًا الجزء الذي يجعل intelligence layer عضو رقم 4 هو المسؤول عن مجرد LLM call مباشر.

هو بنى:

- study material ingestion pipeline
- text extraction
- chunking pipeline
- embeddings generation
- vector retrieval
- RAG logic
- source attribution
- structured citations output

هذا هو العضو الذي جعل السؤال:

"اشرحلي المحاضرة"

يُجاب عليه من ملفات الطالب فعلًا، وليس من معرفة عامة فقط.

2. How He Built It (Technical Deep Dive)

تقنيًا اعتمد على:

- `FastAPI`
- `Supabase PostgreSQL`
- `pgvector`
- `Supabase Storage`
- embedding model
- LLM orchestration
- ملفات backend مثل:
  - `rag_agent.py`
  - `study_materials.py`
  - `embeddings_service.py`
  - `chat.py`
  - `chat_models.py`

هنا كان مبني على مبدأ architecture الـ:

`upload -> extract -> chunk -> embed -> store -> retrieve -> prompt -> answer`

الخطوات التقنية بالتفصيل:

1. عندما الطالب يرفع ملف:
    - backend يتحقق من identity
    - Supabase Storage يخزن الملف في
    - document metadata record ينشئ
2. backend يستخرج النص:
    - PDF عبر parser من
    - Markdown/TXT مباشرة أو من
3. النص يتم تنظيفه وتقسيمه إلى chunks.
4. كل chunk يتم تحويله إلى embedding vector.
5. كل chunk يتم تخزينه في `document_chunks` مع:
    - chunk text
    - embedding
    - document_id
    - course_id
    - uploaded_by
    - metadata إضافية
6. وقت السؤال:
    - سؤال المستخدم أيضًا يتحول إلى embedding
    - يتم عمل similarity search على `pgvector`
    - يتم فلترة النتائج حسب:
        - المستخدم
        - المادة
        - الملفات المختارة
7. أفضل chunks تُبنى داخل prompt موجه
8. الـ LLM ينتج answer grounded + sources

هذا العضو اتعامل أيضًا مع مشاكل حقيقية مثل:

- عدم وجود `filter` parameter في Supabase RPC الحالية
- مباشرة `document_chunks` من fallback retrieval
- mismatch بين `document_id` وtitle
- حالات vector search التي لا تعيد نتائج مناسبة رغم وجود ملفات

يعني دوره لم يكن فقط تنفيذ pipeline، بل جعلها robust.

3. Real Flow Example

مثال حقيقي:

طالب رفع ملف PDF لمحاضرة AI225.

1. الملف يصل إلى `/api/study-materials/upload`.

2. backend يخزن الملف في Supabase Storage.
3. ينشئ record في `documents` .
4. يستخرج النص من الملف.
5. يقسم النص إلى chunks.
6. ينشئ embeddings لكل chunk.
7. يخزن chunks في `document_chunks` .
8. لاحقًا عندما الطالب يسأل:
    - "لخصلي أهم الأفكار"
9. backend:
    - ينشئ embedding للسؤال
    - يبحث في `document_chunks`
    - يختار أفضل المقاطع
    - يبني prompt
    - يستدعي الـ LLM
10. يعيد:

- answer
- sources
- excerpt لكل source

4. Integration With Other Members

هذا العضو يتكامل أساسًا مع:

- Member 3:
    - يستقبل منه user question + course context + selected docs
    - يعيد له answer structured
- Member 2:
    - يعتمد على student academic mapping لمعرفة course ownership
- Member 5:
    - يتفق معه على API contracts والـ backend consistency
- Member 1:
    - بشكل غير مباشر لأن النتيجة النهائية تعرض في UI layer

هو أيضًا يتعامل مباشرة مع:

- database layer
- storage layer
- AI layer

5. Why This Design

اختيار RAG بدل direct LLM call كان قرارًا مهمًا جدًا لعدة أسباب:

- تقليل hallucination
- جعل الإجابة مرتبطة بالمادة الحقيقية
- تمكين citation-based learning
- جعل النظام قابل للثقة أكاديميًا

واستخدام `pgvector` داخل PostgreSQL عبر Supabase كان مناسبًا لأن:

- يحافظ على data stack موحد
- يسهل الربط بين relational data وvector data
- يقلل complexity بدل استخدام vector DB منفصل

Member 5: Academic Intelligence / Planner / Analytics / QA & Integration

1. What He Built

أي الجزء الذي يحول بيانات الطالب إلى RAG، عضو رقم 5 كان مسؤول عن العقل التحليلي للنظام خارج الـ planning وalerts وinsights logic.

هو بنى:

- dashboard backend metrics
- advisor insights
- profile freshness logic
- planner support
- GPA and timeline integration
- integration QA بين الموديولات
- متسقة frontend/backend التحقق من أن العقود بين

بنى Member 5 للمذاكرة، فـ AI intelligence بنى Member 4، وraw student data بنى Member 2 بمعنى أوضح: لو academic intelligence الخاصة بالتخطيط والتحليل.

2. How He Built It (Technical Deep Dive)

اعتمد على:

- Next.js API routes أو backend-style routes داخل frontend app
- Supabase queries
- helper logic مثل `advisor.ts`

- APIs مثل:
  - `/api/dashboard`
  - `/api/advisor`
  - `/api/profile-freshness`

الخطوات التقنية:

1. قراءة بيانات الطالب من:
   - profile
   - student_courses
   - semesters
   - courses
2. حساب metrics مثل:
   - completed credits
   - active courses
   - required credits
   - graduation progress
   - estimated graduation direction
3. تنفيذ freshness checks مثل:
   - هل البروفايل ناقص؟
   - هل يوجد academic history؟
   - هل يوجد ترم أحدث في النظام؟
   - هل توجد مواد حالية؟
   - هل الوقت مناسب للتخطيط للترم القادم؟
4. إرجاع alerts على منظمة تحتوي:
   - title
   - message
   - tone
   - CTA label
   - CTA href

هذا العضو أيضًا كان مسؤولًا عن نقطة مهمة جدًا: منع اختلاط responsibilities بين UI وlogic.

فمثلًا:

- Member 1 يعرض alert
- ولماذا وإلى أين يوجّه المستخدم alert هو الذي يحدد متى يظهر Member 5 لكن

3. Real Flow Example

مثال عملي:

الطالب يفتح الداشبورد.

1. الواجهة تطلب `/api/profile-freshness` .
2. الـ API تقرأ:
    - profile
    - student_courses
    - latest user semester
    - latest system semester
3. لو وجدت أن الطالب لديه current semester مسجل لكن لا توجد مواد planned للترم القادم:
    - تُنشئ alert من نوع planning suggestion
4. الـ frontend يعرض التنبيه:
    - `Good time to plan ahead`
    - CTA مثل `Plan next term`
5. عند الضغط، ينتقل المستخدم إلى Planner

مثال آخر:

لو الأكاديميك history غير مكتمل:

1. freshness API ترصد النقص
2. ترجع alert مناسب
3. الواجهة تعرض زر يوجّه المستخدم إلى Profile بدل Planner

4. Integration With Other Members

هذا العضو يتكامل مع الجميع تقريبًا:

- من Member 2:
    - يستقبل profile data وacademic history
- إلى Member 1:
    - يرسل dashboard-ready insights وalerts
- مع Member 3:
    - مرتبط بالواقع الأكاديمي Study Chat يوفر سياق عام يجعل
- مع Member 4:
    - عند الحاجة chat/backend flows في contracts ينسق

كذلك هذا العضو يقوم بدور `integration owner` ، أي أنه يراجع:

- هل الـ payload صحيح؟
- هل الـ route يعيد expected response؟
- هل التغيير في موديول معيّن كسر موديول آخر؟

5. Why This Design

التصميم ده اتختار لأن النظام لا يجب أن يكون مجرد store للبيانات، بل يجب أن يحول البيانات إلى قرارات.

مميزات هذا الأسلوب:

- dashboard يصبح proactive وليس passive
- alerts تصبح مبنية على data حقيقية
- profile التخطيط الأكاديمي يصبح امتداد طبيعي للـ
- frontend لا يتحمل منطق حسابي معقد

كذلك فصل analytics APIs في ownership مستقل يقلل:

- تداخل المسؤوليات
- تكرار الحسابات في أكثر من مكان
- أخطاء التكامل

Full System Integration Flow

لفهم المشروع بشكل كامل، لازم نشوفه كمنظومة واحدة، وليس كخمس أجزاء منفصلة.

الـ integration الكامل يمشي كالتالي:

1. الطالب يسجل الدخول عبر Supabase Auth.
2. النظام يحصل على access token صالح.
3. الواجهة تحمل الصفحات المحمية.
4. Member 2's modules توفر بيانات profile وacademic history.
5. Member 5's APIs تحول هذه البيانات إلى:
    - dashboard metrics
    - alerts
    - advisor-like insights
6. Member 1's UI تعرض هذه البيانات في شكل واضح ومقروء.
7. عندما يدخل الطالب إلى Study Chat:
    - Member 3's frontend يحمّل المواد الحالية
    - ويعرض مكتبة المصادر
8. إذا رفع الطالب ملفًا:
    - Member 4's ingestion pipeline تخزن الملف
    - تستخرج النص
    - تنشئ chunks وembeddings
    - تحفظها في database/vector layer
9. عندما يسأل الطالب سؤالًا:
    - Member 3 يرسل payload منظم
    - Member 4 ينفذ retrieval + prompt + answer generation
    - Member 4 يعيد citations
    - Member 3 يعرض الإجابة والمصادر
10. Member 1 بناها UI وتجربة shell كل ذلك يظهر داخل

11. والمنطق العام متسقين مع حالة الطالب الحقيقية planner والـ alerts يتأكد أن Member 5 ،وفي الخلفية

بالتالي النظام يعمل كسلسلة مترابطة:

`User -> Auth -> Frontend UI -> API layer -> Database / RAG -> Response -> UI`

System Intelligence Explanation

داخل النظام RAG كيف يعمل

RAG اختصار لـ `Retrieval-Augmented Generation`.

يجيب من ذاكرته العامة فقط، ولكن LLM في هذا المشروع، الفكرة ليست أن الـ:

1. المستخدم يرفع ملفات مرتبطة بمادة معيّنة.
2. النظام يحول هذه الملفات إلى chunks صغيرة مفهومة.
3. كل chunk يتحول إلى embedding vector.
4. عندما يكتب الطالب سؤالًا:
    - السؤال أيضًا يتحول إلى embedding
5. يتم البحث عن أقرب chunks دلاليًا داخل `pgvector`.
6. هذه الـ chunks فقط تُعطى للـ LLM داخل prompt.
7. الـ LLM يبني الإجابة اعتمادًا على تلك المقاطع المسترجعة.

إذًا الـ LLM لا يعمل في فراغ، بل يعمل فوق context محدد ومفلتر.

لماذا نستخدم embeddings + vector search

السبب هو أن البحث التقليدي بالكلمات المفتاحية فقط لا يكفي.

مثال:

لو الطالب سأل:

"اشرحلي أهم أفكار المحاضرة الأولى"

قد لا تكون نفس الجملة موجودة حرفيًا داخل الملف، لكن semantic meaning قريب من:

- introduction
- key concepts
- foundations

هنا يأتي دور embeddings:

- تحويل النص إلى تمثيل رقمي دلالي
- تمكين similarity search على مستوى المعنى لا الكلمة فقط

وده مهم جدًا في المواد الأكاديمية لأن الطالب قد يصيغ السؤال بطريقة مختلفة تمامًا عن صياغة الكتاب أو السلايد.

كيف يتم تقليل الـ Hallucination

النظام يقلل hallucination بعدة طرق:

1. `Course scoping`
   - السؤال مربوط بمادة محددة
2. `User scoping`
   - النتائج مربوطة بملفات الطالب أو السياق المسموح
3. `Selected document filtering`
   - الطالب يقدر يحدد ملفًا بعينه
4. `RAG prompt grounding`
   - الـ LLM يتلقى chunks مسترجعة فعلًا من المصادر
5. `Citation rendering`
   - كل إجابة تعرض مصادرها
6. `Fallback retrieval logic`
   - لو vector filtering لم يرجع chunk النظام لديه fallback مناسب،

هذا يجعل الإجابة:

- أكثر دقة
- أكثر شفافية
- أسهل في التحقق

وده مهم جدًا لأننا نتعامل مع studying context، وليس دردشة عامة فقط.

Viva Ready Summary

لو أحد سأل في المناقشة:

"عندكم system design اشرحوا تقسيم الفريق والـ

فالإجابة القوية تكون كالتالي:

المشروع اتقسم إلى خمس طبقات واضحة ownership.
البيانات الأكاديمية الأساسية، وعضو profile/onboarding وواجهة الاستخدام، وعضو مسؤول عن dashboard عضو مسؤول عن الـ vector والـ embeddings والـ RAG والـ AI backend وتجربة التعلم، وعضو مسؤول عن Study Chat frontend مسؤول عن retrieval، وعضو مسؤول عن dashboard analytics والplanner والalerts والتكامل بين الموديولات.

المعمارية نفسها قائمة على:

- `Next.js` للواجهة
- `FastAPI` لـ AI/backend endpoints
- `Supabase PostgreSQL` للبيانات
- `pgvector` لـ embeddings search
- `Supabase Storage` للملفات

ثم استرجاع الأجزاء، embeddings تحويلها إلى chunks، حيث يتم رفع ملفات المادة، تقسيمها إلى RAG، أما الذكاء في النظام فمبني على ملفات حقيقية وليست مجرد تخمين عام grounded وبهذا تكون الإجابة .LLM الأكثر صلة بالسؤال قبل إرسالها إلى الـ.

واضحة، API contracts واضح، و data model مبني فوق feature ولكن في أن كل، features قوة المشروع ليست فقط في وجود عدة ownership boundaries و تقلل التداخل وتزيد من جودة التكامل.

خاتمة

هذا التقسيم يوضح أن كل عضو لم يعمل في عزلة، بل كان كل جزء مبني بحيث يخدم جزءًا آخر:

- profile يغذي dashboard
- dashboard يوجه الطالب
- Study Chat يعتمد على مواد الطالب الحالية
- RAG يعتمد على الـ uploads والـ vector storage
- planner وalerts يعتمدان على consistency البيانات

وبالتالي المشروع كله يمثل نظامًا متكاملًا يجمع بين:

- academic record management
- intelligent analytics
- AI-powered studying
- maintainable modular architecture